InteliScrap AI — Development Blueprint

5-Man Dev Task Allocation, Database Architecture & API Specifications

Team:	Team Nexus	**Project Type:**	Offline-First Edge ML Mobile PWA
Target Model:	Gemma 4 (E2B / E4B quantized edge weights)	**Backend Stack:**	FastAPI + PostgreSQL + Railway

To ensure a seamless development experience during the **Build with Gemma Hackathon**, Team Nexus must adopt an aggressive, specialized task division structure. This avoids bottlenecking and minimizes merge conflicts. Because InteliScrap AI utilizes a hybrid architecture—combining high-complexity client-side edge machine learning with a traditional FastAPI relational backend—each of the 5 team members is allocated an explicit, decoupled subsystem to own.

1. Team Nexus Task Distribution

The table below details the 5 specific roles, ownership, core responsibilities, and deliverables. This ensures that everyone knows exactly what they are building, allowing parallel development.

Role & Owner	Core Subsystem & Code Ownership	Key Hackathon Deliverables
Member 1 **Lead Backend & Database Engineer**	**Core Cloud Backend Engine** Owns the FastAPI server framework, SQLAlchemy schemas, database migration patterns, and the critical server synchronization endpoint. *Files:* `models.py`, `database.py`, `schemas.py`, `sync_api.py`, `main.py`	• Working API endpoints on Railway • PostgreSQL schema setup • Bidirectional sync handling
Member 2 **Frontend & PWA Architect**	**Offline-First Shell & UX** Responsible for React UI components, CSS design matching mobile dimensions, PWA Service Worker configuration, and local indexedDB transactions. *Files:* `App.jsx`, `service-worker.js`, `db.js`, `CameraCapture.jsx`	• High-fidelity React application • Service worker assets caching • Dexie.js (IndexedDB) state
Member 3 **Edge AI & Local ML Engineer**	**On-Device Gemma 4 Integration** Owns on-device model initialization via WebLLM/MediaPipe, passing local camera video frames, writing strict output prompt contracts, and enforcing JSON responses. *Files:* `useGemma.js`, `promptTemplates.js`, `imageUtils.js`	• Gemma 4 model downloading • Structured offline inference • Image-to-text token processing
Member 4 **Accessibility & Audio Engineer**	**Multilingual Voice Synthesis** Builds the offline Text-to-Speech phonetic audio loop to vocalize grading and safety outcomes in spoken Hausa and Pidgin without internet. *Files:* `useAudioTTS.js`, `locales/ha.json`, `locales/pcm.json`	• Phonetic native translation matrices • Web Speech local voice triggers • Accessible high-contrast UX
Member 5 **QA, DevOps & Sync Engineer**	**Infrastructure & Integrity** Owns deployment scripts, offline-online sync conflict logic, testing offline edge-cases, Docker build processes, and the final Kaggle documentation. *Files:* `syncConflict.js`, `Dockerfile`, `tests/`, `railway.json`	• Dockerized multi-stage container • Automated API testing suite • Smooth sync conflict handler

> 💡 **Maximizing Dev Speed: Decouple early!**
>
> Member 2 (Frontend) and Member 3 (Edge AI) can develop entirely on local static assets before Member 1 (Backend) finishes the FastAPI service. Member 1 and Member 5 must mock API payloads so Member 2 can safely test backend interactions without waiting.

2. Technical Database & API Specifications

The following architectural blueprints specify exactly what the engineering teammates will develop. Follow these specifications precisely to maintain absolute compatibility during final integration.

A. Database Models (Relational Schema — Member 1)

The system database is relational, deployed on PostgreSQL. It handles multi-user authentication, records the transactional history of processed scrap materials, and acts as the central reference for current regional scrap prices.

`models.py (SQLAlchemy Schema Blueprint)`

```
from sqlalchemy import Column, String, Integer, Float, Boolean, DateTime, ForeignKey, JSON
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
from .database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(String, primary_key=True, index=True) # UUID/Cognito ID
    phone_number = Column(String, unique=True, nullable=False, index=True)
    full_name = Column(String, nullable=True)
    location_hub = Column(String, default="Zaria")
    created_at = Column(DateTime(timezone=True), server_default=func.now())

    scans = relationship("ScrapScan", back_populates="owner")

class ScrapScan(Base):
    __tablename__ = "scrap_scans"

    id = Column(String, primary_key=True, index=True) # UUID generated offline
    user_id = Column(String, ForeignKey("users.id"), nullable=False)
    material_class = Column(String, nullable=False) # e.g., Copper, Lead-Acid Battery
    sub_grade = Column(String, nullable=True)       # e.g., Grade A, Wire-copper
    weight_est_kg = Column(Float, nullable=True)
    estimated_naira_value = Column(Float, nullable=False)
    confidence_score = Column(Float, default=1.0)

    # Metadata extracted by Gemma 4 edge model
    toxicity_hazards = Column(JSON, nullable=True)  # List of hazards e.g., ["acid_burn",
"lead_fumes"]
    safety_instructions = Column(String, nullable=True)

    # Tracking offline status
    captured_at = Column(DateTime, nullable=False)
    synced_at = Column(DateTime(timezone=True), server_default=func.now())
    is_deleted = Column(Boolean, default=False)

    owner = relationship("User", back_populates="scans")

class PriceMatrix(Base):
    __tablename__ = "price_matrices"

    id = Column(Integer, primary_key=True, autoincrement=True)
    material_class = Column(String, unique=True, index=True, nullable=False)
    price_per_kg_naira = Column(Float, nullable=False)
    last_updated = Column(DateTime(timezone=True), onupdate=func.now(),
server_default=func.now())
    updated_by = Column(String, nullable=True)
```

B. Bidirectional Sync API (FastAPI Endpoint — Member 1 & Member 5)

The sync endpoint must accept an array of local, offline-generated scan records, process updates against potential conflict states, write safely to the database, and return the latest market prices to keep the client device up-to-date.

`sync_api.py (FastAPI Routing Blueprint)`

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List
from datetime import datetime
from .database import get_db
from . import schemas, models

router = APIRouter(prefix="/api/v1", tags=["sync"])

@router.post("/sync", response_model=schemas.SyncResponse)
def synchronize_client(
    payload: schemas.SyncPayload,
    db: Session = Depends(get_db)
):
    # 1. Process client local scans
    synced_scan_ids = []

    for client_scan in payload.local_scans:
        # Check if record already exists to avoid duplication (Idempotency)
        existing = db.query(models.ScrapScan).filter(models.ScrapScan.id ==
client_scan.id).first()

        if existing:
            # Simple conflict resolution: Local updates take precedence if newer
            if client_scan.captured_at > existing.captured_at:
                existing.material_class = client_scan.material_class
                existing.estimated_naira_value = client_scan.estimated_naira_value
                existing.toxicity_hazards = client_scan.toxicity_hazards
                existing.is_deleted = client_scan.is_deleted
                existing.captured_at = client_scan.captured_at
            synced_scan_ids.append(existing.id)
        else:
            # Create new scan
            new_scan = models.ScrapScan(
                id=client_scan.id,
                user_id=payload.user_id,
                material_class=client_scan.material_class,
                sub_grade=client_scan.sub_grade,
                weight_est_kg=client_scan.weight_est_kg,
                estimated_naira_value=client_scan.estimated_naira_value,
                confidence_score=client_scan.confidence_score,
                toxicity_hazards=client_scan.toxicity_hazards,
                safety_instructions=client_scan.safety_instructions,
                captured_at=client_scan.captured_at,
                is_deleted=client_scan.is_deleted
            )
            db.add(new_scan)
            synced_scan_ids.append(new_scan.id)

    db.commit()

    # 2. Retrieve the latest Price Matrix to update client offline cache
```

```
    prices = db.query(models.PriceMatrix).all()
    price_matrix_list = [
        schemas.PriceMatrixItem(
            material_class=p.material_class,
            price_per_kg_naira=p.price_per_kg_naira
        ) for p in prices
    ]

    return {
        "status": "success",
        "synced_ids": synced_scan_ids,
        "latest_prices": price_matrix_list,
        "server_time": datetime.utcnow()
    }
```

3. Client-Side Edge AI & Local Storage Blueprints

The core novelty of InteliScrap AI is its ability to perform high-fidelity image assessment and speech synthesis completely without internet. The following modules show the interaction between client storage and local Gemma 4 weights.

A. IndexedDB Client Schema (Offline Storage — Member 2)

Dexie.js is used to provide a clean, promise-based API over raw IndexedDB. It tracks scans and local cached price matrices.

`db.js (Dexie Client Storage Schema)`

```
import Dexie from 'dexie';

export const db = new Dexie('InteliScrapLocalDB');

// Declare local offline stores
db.version(1).stores({
  scans: 'id, user_id, material_class, estimated_naira_value, captured_at, is_synced,
is_deleted',
  prices: 'material_class, price_per_kg_naira, last_updated'
});
```

B. Local Gemma 4 Model Execution (Edge AI — Member 3)

This react hook handles the local instance of the Gemma 4 vision model running inside the browser engine using WebLLM.

`useGemma.js (Edge Model Execution Hook)`

```
import { useState, useEffect } from 'react';
import { CreateWebWorkerMLCEngine } from '@mlc-ai/web-llm';

export function useGemma() {
  const [engine, setEngine] = useState(null);
  const [loadingProgress, setLoadingProgress] = useState(0);
  const [isReady, setIsReady] = useState(false);

  useEffect(() => {
    async function initGemma() {
      // Initialize model on Web Worker thread to keep main thread completely smooth
      const webWorkerEngine = await CreateWebWorkerMLCEngine(
        new Worker(new URL('./gemmaWorker.js', import.meta.url), { type: 'module' }),
        "Gemma-4-E2B-quantized-edge", // Targeting E2B model package optimized for browser
        {
          initProgressCallback: (report) => {
            setLoadingProgress(Math.round(report.progress * 100));
          }
        }
      );
      setEngine(webWorkerEngine);
      setIsReady(true);
    }
    initGemma();
  }, []);

  const analyzeScrapImage = async (imageBlob) => {
    if (!engine) throw new Error("Gemma engine not ready yet");

    // Convert Image blob to Base64/Tensor expected by local vision-to-text model
    const base64Image = await convertBlobToBase64(imageBlob);

    const systemPrompt = `
      You are an expert heavy metal recycler in Northern Nigeria.
      Analyze the scrap material image and return a strict JSON payload containing:
      - material_class: string (e.g. Copper, Aluminum, Lead-Acid Battery, Lithium-Ion, Brass)
      - confidence: float (0.0 to 1.0)
      - toxicity_hazards: list of strings (e.g. ["corrosive_acid", "lead_poisoning"])
      - safety_instructions: string (short instructions in simple words)
      Output JSON only. Do not wrap in markdown tags or write conversational text.
    `;

    const response = await engine.chat.completions.create({
      messages: [
        { role: "system", content: systemPrompt },
        { role: "user", content: [
            { type: "text", text: "Classify this piece of metal scrap." },
            { type: "image_url", image_url: { url: base64Image } }
          ]
        }
      ],
      response_format: { type: "json_object" } // Enforce strict JSON output
```

```
    });

    return JSON.parse(response.choices[0].message.content);
  };

  return { analyzeScrapImage, loadingProgress, isReady };
}

function convertBlobToBase64(blob) {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onloadend = () => resolve(reader.result);
    reader.onerror = reject;
    reader.readAsDataURL(blob);
  });
}
```

C. Client Offline Multilingual Audio synthesis (Accessibility — Member 4)

The Audio Engine runs fully offline using client speech synthesis. It matches translated warning templates with phonetics in Hausa or Pidgin based on the JSON output from the Gemma 4 model.

`useAudioTTS.js (Offline Audio Hook)`

```
import { useCallback } from 'react';

export function useAudioTTS() {
  const speakReport = useCallback((materialClass, nairaValue, hazards = [], language = 'ha') =>
{
    if (!('speechSynthesis' in window)) {
      console.warn("Speech Synthesis is not supported on this device");
      return;
    }

    // Stop ongoing speech
    window.speechSynthesis.cancel();

    // Multilingual speech string dictionaries using local voice arrays
    let speechString = "";

    if (language === 'ha') { // Hausa
      const hazardPhrase = hazards.length > 0
        ? `Saurara! Wannan kaya yana da hatsari kamar haka: ${hazards.join(', ')}.`
        : "Wannan kayan yana da aminci.";

      speechString = `An gano ${materialClass}. Farashin sa shine naira ${nairaValue} duk kilo.
${hazardPhrase}`;
    } else { // Nigerian Pidgin (Default)
      const hazardPhrase = hazards.length > 0
        ? `Watch out! Dis scrap get danger o: ${hazards.join(', ')}.`
        : "Dis scrap dey very safe to touch.";

      speechString = `We find ${materialClass}. The price na ${nairaValue} Naira per kg. $
{hazardPhrase}`;
    }

    const utterance = new SpeechSynthesisUtterance(speechString);

    // Attempt to target regional phonetic voices local to Chrome/Android WebView
    const voices = window.speechSynthesis.getVoices();
    const targetVoice = voices.find(v => v.lang.startsWith('ha') || v.lang.startsWith('en-
NG'));
    if (targetVoice) {
      utterance.voice = targetVoice;
    }

    utterance.rate = 0.9; // Spoken slightly slower for optimal phonetic comprehension
    window.speechSynthesis.speak(utterance);
  }, []);

  return { speakReport };
}
```

4. Step-by-Step Integration & Deployment Roadmap

To ensure a stress-free hackathon submission, Team Nexus must plan integration meticulously. Avoid merging code at the last minute. Follow this structured roadmap:

- **Milestone 1 (Days 1–2) — Independent Setup:** Member 1 finishes the core FastAPI server and local database. Member 2 spins up the React visual skeleton and caches basic static layouts offline. Member 3 builds a basic worker script to test local WebLLM compiling in the background.
- **Milestone 2 (Day 3) — Interface Connection:** Member 2 imports Dexie database schemas and connects the Camera UI capture element. Member 3 hooks the camera stream up to `useGemma.js`, confirming that images are processed into structured JSON outputs. Member 4 finishes audio phonetic arrays and binds them to Gemma's output triggers.
- **Milestone 3 (Day 4) — Local Sync & Conflict Resolution:** Member 5 creates the conflict logic scripts (e.g., verifying timestamp differentials if user scans the same material item multiple times). Member 1 verifies backend sync integration locally using simulated network loss inside Postman.
- **Milestone 4 (Day 5) — Cloud Deploy & Submission:** Member 5 containerizes the backend FastAPI app via Docker and deploys it on Railway. The frontend is published to Vercel or GitHub Pages. The team tests the entire flow from camera snap to offline indexedDB writes, to network reconnection, to DB synchronization on the live cloud. Write the Kaggle documentation as a team and hit submit!